101 PYTHON MINI PROJECTS

FULL SOURCE CODE • PROJECT-LEVEL BUILDS • BEGINNER → ADVANCED

A practical project book focused on applications, tools, APIs, automation, data, AI, games and management systems — not one-function beginner exercises.

Each project includes a complete starter implementation rather than a tiny syntax exercise.

Project Index

#	Project	Focus
1	Password Manager Vault	CLI app that creates a local encrypted-style vault using PBKDF2 + Fernet.
2	Personal Expense Tracker	SQLite-backed expense tracker with categories, monthly totals and CSV export.
3	File Organizer Pro	Desktop folder organizer with extension rules, duplicate detection and dry-run...
4	PDF Invoice Generator	Generate professional invoices from a JSON order file.
5	Quiz Master App	Timed JSON-driven quiz with scoring, review and leaderboard storage.
6	URL Shortener Service	Flask API that creates short links and tracks click counts.
7	Weather Dashboard	CLI weather dashboard using Open-Meteo with caching and forecast summaries.
8	Contact Book with Search	SQLite contact manager with tags, search, update and JSON backup.
9	Study Planner	Priority-based study planner with deadlines, completion tracking and daily age...
10	Inventory Management System	SQLite inventory with stock alerts, purchase entries and sales deductions.
11	Pomodoro Productivity Desktop	GUI productivity timer with session history.
12	Markdown Notes Manager	Searchable markdown notebook with tags and index.
13	Habit Tracker	SQLite habit tracker with streak calculations.
14	Banking Simulator	Account management, transfers, transaction history and statements.
15	Library Management System	Books, members, lending, returns and overdue reports.
16	Hotel Booking Manager	Rooms, reservations, check-in/check-out and billing.
17	Cinema Ticket Booking	Seat map, bookings, cancellation and receipt generation.
18	Restaurant POS	Menu, cart, taxes, discounts and daily sales report.
19	Invoice & GST Calculator	Invoice builder with Indian GST breakdown and PDF export.
20	Attendance Management	Student attendance database with percentage reports.
21	Employee Payroll System	Salary calculation, deductions, payslip generation.
22	Resume Builder	JSON-driven resume builder exporting polished PDF.
23	Job Application Tracker	Track companies, stages, deadlines and follow-ups.
24	Interview Question Bank	Searchable categorized interview questions with quiz mode.
25	DSA Practice Tracker	Track problems, difficulty, patterns and revision dates.
26	Code Snippet Manager	Tagged snippet vault with full-text search.
27	Git Commit Analyzer	Analyze a local Git repository and produce contribution reports.
28	Log File Analyzer	Parse application logs and generate error summaries.
29	Website Uptime Monitor	Periodically check URLs and save availability history.
30	Broken Link Checker	Crawl a website and report broken internal links.
31	Website Sitemap Generator	Crawl internal pages and create sitemap.xml.
32	Web Scraper to CSV	Extract structured product/article data into CSV.
33	News Aggregator	Fetch RSS feeds, deduplicate and categorize headlines.
34	Email Newsletter Builder	Build HTML newsletters from templates and data.
35	QR Code Event Check-in	Generate attendee QR codes and scan/check-in records.
36	PDF Merger & Splitter	GUI tool for merging and splitting PDF documents.
37	Image Batch Processor	Resize, watermark and compress folders of images.
38	Duplicate Photo Finder	Perceptual hashing to find visually similar photos.
39	Video Metadata Catalog	Catalog video metadata and searchable library.
40	Audio Playlist Manager	Manage playlists and generate M3U files.
41	YouTube Download Queue	Queue public video URLs for local processing.
42	Personal Knowledge Base	Local searchable notes with backlinks.
43	Password Strength Auditor	Score passwords locally with entropy rules.
44	OTP Authenticator Demo	Generate TOTP codes for locally stored demo secrets.
45	Secure File Locker	Encrypt/decrypt user-selected files with a password.
46	Clipboard History Manager	Desktop clipboard history with search.
47	Screenshot Organizer	Capture screenshots and auto-classify by date.
48	Desktop Notification Scheduler	Schedule local reminders and notifications.
49	System Resource Dashboard	Monitor CPU, RAM, disk and network usage.
50	Process Manager	List, filter and safely request termination of processes.

#	Project	Focus
51	Disk Usage Analyzer	Recursive folder size analyzer with largest-items report.
52	Backup Manager	Versioned folder backups with retention policy.
53	Folder Sync Tool	One-way sync with hashes and dry-run preview.
54	CSV Cleaner	Detect missing values, duplicates and malformed rows.
55	Excel Report Generator	Create formatted Excel reports from CSV data.
56	PDF Report Generator	Turn analytics data into a formatted PDF report.
57	Sales Dashboard	Plot sales trends, regions and top products.
58	Student Performance Dashboard	Analyze marks, grades and subject trends.
59	Loan EMI Planner	Calculate EMI, amortization schedule and payoff scenarios.
60	Investment Portfolio Tracker	Track holdings, average cost and P/L from CSV prices.
61	Crypto Portfolio Simulator	Paper portfolio with simulated trades and P/L.
62	Currency Converter	Live exchange-rate converter with cached rates.
63	Flight Fare Watcher	Store target routes and compare fetched public fare data.
64	Product Price Tracker	Track public product prices from supplied pages.
65	Book Recommendation Engine	Content-based recommendation from book metadata.
66	Movie Recommendation Engine	Genre/tag similarity recommendation system.
67	Music Recommendation Engine	Similarity-based playlist generator.
68	Resume Keyword Matcher	Compare a resume against a job description.
69	Cover Letter Generator	Template-driven cover letters from structured data.
70	Text Summarizer	Extractive summarizer for local text documents.
71	PDF Text Search Engine	Index PDF text and search across a local collection.
72	OCR Document Pipeline	OCR image documents and save searchable text.
73	Language Detector	Detect language from text using a lightweight model.
74	Sentiment Analyzer	Classify review sentiment with TF-IDF + logistic regression.
75	Spam Email Classifier	Train a local text classifier on labeled emails.
76	House Price Predictor	Regression model with preprocessing and evaluation.
77	Student Score Predictor	Regression model with train/test evaluation.
78	Customer Churn Predictor	Classification pipeline with metrics and feature importance.
79	Loan Approval Predictor	Classification demo with preprocessing and metrics.
80	Sales Forecasting Tool	Time-series baseline forecast and visualization.
81	Anomaly Detector	Detect unusual numeric observations with robust statistics.
82	Image Classifier	Train a small CNN on a folder-based image dataset.
83	Face Attendance Demo	Face embeddings for local attendance demonstration.
84	Hand Gesture Controller	Recognize basic hand gestures with webcam input.
85	Object Detection Demo	Run a pretrained detector on webcam frames.
86	Voice Notes App	Record microphone audio and create timestamped notes.
87	Speech-to-Text Journal	Transcribe microphone input into daily journal files.
88	Text-to-Speech Reader	Read documents aloud with playback controls.
89	AI Chat Assistant	Local chat UI with pluggable model/API backend.
90	Document Q&A Assistant	Chunk local documents and retrieve relevant passages.
91	Code Explanation Assistant	Generate structured explanations for pasted code.
92	SQL Query Assistant	Convert natural-language questions into reviewed SQL.
93	AI Study Buddy	Generate quizzes from user-provided study notes.
94	Flashcard Generator	Turn notes into structured flashcards.
95	Timetable Generator	Constraint-based class timetable generator.
96	Pathfinding Visualizer	Visualize BFS, DFS and A* on a grid.
97	Sorting Visualizer	Animate multiple sorting algorithms with metrics.
98	Tic-Tac-Toe AI	Minimax-based unbeatable Tic-Tac-Toe.
99	Snake Game	Pygame snake with levels, score and saved high score.
100	Flappy Bird Clone	Pygame arcade game with collision and score.
101	Space Shooter	Pygame shooter with waves, power-ups and scoring.

1. Password Manager Vault

CLI app that creates a local encrypted-style vault using PBKDF2 + Fernet.

```
import os, json, base64, getpass
from cryptography.fernet import Fernet
from hashlib import pbkdf2_hmac

FILE = "vault.json"

def key_from_password(password, salt):
    raw = pbkdf2_hmac("sha256", password.encode(), salt, 200_000, 32)
    return base64.urlsafe_b64encode(raw)

def load():
    if not os.path.exists(FILE): return None
    with open(FILE) as f: return json.load(f)

master = getpass.getpass("Master password: ")
data = load()
if data:
    salt = base64.b64decode(data["salt"])
    key = key_from_password(master, salt)
    f = Fernet(key)
    try:
        records = json.loads(f.decrypt(data["blob"].encode()).decode())
    except Exception:
        raise SystemExit("Wrong password or damaged vault.")
else:
    salt = os.urandom(16); key = key_from_password(master, salt)
    f = Fernet(key); records = {}

while True:
    print("\n1.Add  2.List  3.Get  4.Delete  5.Exit")
    choice = input("Choice: ")
    if choice == "1":
        site = input("Site: "); user = input("Username: ")
        pwd = getpass.getpass("Password: ")
        records[site] = {"username": user, "password": pwd}
    elif choice == "2":
        print(*records.keys(), sep="\n")
    elif choice == "3":
        site = input("Site: "); print(records.get(site, "Not found"))
    elif choice == "4":
        records.pop(input("Site: "), None)
    elif choice == "5":
        blob = f.encrypt(json.dumps(records).encode()).decode()
        with open(FILE, "w") as fp:
            json.dump({"salt": base64.b64encode(salt).decode(), "blob": blob}, fp)
        break
```

2. Personal Expense Tracker

SQLite-backed expense tracker with categories, monthly totals and CSV export.

```
import sqlite3, csv
from datetime import date

db = sqlite3.connect("expenses.db")
db.execute("""CREATE TABLE IF NOT EXISTS expenses(
 id INTEGER PRIMARY KEY, day TEXT, category TEXT, note TEXT, amount REAL)""")
db.commit()

def add():
    day = input("Date YYYY-MM-DD [today]: ") or str(date.today())
    cat = input("Category: "); note = input("Note: ")
    amount = float(input("Amount: "))
    db.execute("INSERT INTO expenses(day,category,note,amount) VALUES(?,?,?,?)",
               (day,cat,note,amount)); db.commit()

def report():
    month = input("Month YYYY-MM: ")
    rows = db.execute("""SELECT category,SUM(amount)
        FROM expenses WHERE day LIKE ? GROUP BY category ORDER BY 2 DESC""", (month+"-%",)).
fetchall()
    print("\n".join(f"{c:15} ■{a:.2f}" for c,a in rows))
    print("Total:", db.execute("SELECT COALESCE(SUM(amount),0) FROM expenses WHERE day LIKE ?",
                               (month+"-%",)).fetchone()[0])

def export():
    rows = db.execute("SELECT day,category,note,amount FROM expenses").fetchall()
    with open("expenses.csv","w",newline="") as f:
        w=csv.writer(f); w.writerow(["date","category","note","amount"]); w.writerows(rows)

while True:
    print("\n1 Add  2 Report  3 Export  4 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": report()
    elif c=="3": export(); print("Exported.")
    elif c=="4": break
```

3. File Organizer Pro

Desktop folder organizer with extension rules, duplicate detection and dry-run mode.

```
from pathlib import Path
import hashlib, shutil

ROOT = Path(input("Folder to organize: ")).expanduser()
DRY_RUN = input("Dry run? y/n: ").lower() == "y"
rules = {
    "Images": {".jpg",".jpeg",".png",".gif",".webp"},
    "Documents": {".pdf",".docx",".doc",".txt",".xlsx",".csv"},
    "Videos": {".mp4",".mkv",".mov",".avi"},
    "Audio": {".mp3",".wav",".flac"},
    "Archives": {".zip",".rar",".7z",".tar",".gz"},
}

def digest(path):
    h=hashlib.sha256()
    with open(path,"rb") as f:
        for chunk in iter(lambda:f.read(1024*1024),b""): h.update(chunk)
    return h.hexdigest()

seen={}
for p in ROOT.iterdir():
    if not p.is_file(): continue
    folder=next((k for k,v in rules.items() if p.suffix.lower() in v),"Others")
    target=ROOT/folder; target.mkdir(exist_ok=True)
    key=(p.stat().st_size,digest(p))
    if key in seen:
        print("Duplicate:",p,"=>",seen[key])
        continue
    seen[key]=p
    dest=target/p.name
    print(p,"->",dest)
    if not DRY_RUN:
        shutil.move(str(p),str(dest))
print("Done.")
```

4. PDF Invoice Generator

Generate professional invoices from a JSON order file.

```
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Table, TableStyle, Spacer
from reportlab.lib import colors
import json

with open("order.json") as f: order=json.load(f)
doc=SimpleDocTemplate("invoice.pdf",pagesize=A4,rightMargin=36,leftMargin=36,topMargin=36,
bottomMargin=36)
story=[Paragraph("<b>INVOICE</b>", __import__("reportlab.lib.styles",fromlist=[
"getSampleStyleSheet"]).getSampleStyleSheet()["Title"]),
       Paragraph(f"Customer: {order['customer']}", __import__("reportlab.lib.styles",fromlist=[
"getSampleStyleSheet"]).getSampleStyleSheet()["Normal"]), Spacer(1,15)]
rows=[["Item","Qty","Price","Amount"]]
total=0
for x in order["items"]:
    amount=x["qty"]*x["price"]; total+=amount
    rows.append([x["name"],str(x["qty"]),f"■{x['price']:.2f}",f"■{amount:.2f}"])
rows.append(["","","TOTAL",f"■{total:.2f}"])
t=Table(rows,colWidths=[230,50,80,90])
t.setStyle(TableStyle([("BACKGROUND",(0,0),(-1,0),colors.black),
("TEXTCOLOR",(0,0),(-1,0),colors.white),("GRID",(0,0),(-1,-1),.5,colors.grey),
("ALIGN",(1,1),(-1,-1),"RIGHT"),("FONTNAME",(0,0),(-1,0),"Helvetica-Bold")]))
story += [t, Spacer(1,20), Paragraph("Thank you for your business.", __import__("reportlab.lib.
styles",fromlist=["getSampleStyleSheet"]).getSampleStyleSheet()["Normal"])]
doc.build(story)
print("invoice.pdf created")
```

5. Quiz Master App

Timed JSON-driven quiz with scoring, review and leaderboard storage.

```
import json, random, time
from pathlib import Path

questions=json.load(open("questions.json"))
random.shuffle(questions)
score=0; wrong=[]
start=time.time()
for i,q in enumerate(questions,1):
    print(f"\n{i}/{len(questions)}  {q['question']}")
    for j,opt in enumerate(q["options"],1): print(j,opt)
    try: ans=int(input("Answer: "))
    except: ans=0
    if ans == q["answer"]: score += 1
    else: wrong.append(q)
elapsed=time.time()-start
name=input("\nYour name: ")
board=[]
if Path("leaderboard.json").exists():
    board=json.load(open("leaderboard.json"))
board.append({"name":name,"score":score,"time":round(elapsed,1)})
board=sorted(board,key=lambda x:(-x["score"],x["time"]))[:10]
json.dump(board,open("leaderboard.json","w"),indent=2)
print(f"Score: {score}/{len(questions)} | Time: {elapsed:.1f}s")
print("\nTop players:")
for x in board: print(x["name"],x["score"],x["time"])
```

6. URL Shortener Service

Flask API that creates short links and tracks click counts.

```
from flask import Flask, request, redirect, jsonify
import secrets, sqlite3, datetime

app=Flask(__name__)
db=sqlite3.connect("links.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS links(code TEXT PRIMARY KEY,url TEXT,clicks INTEGER DEFAULT
0)")
db.commit()

@app.post("/shorten")
def shorten():
    url=request.json.get("url","")
    code=secrets.token_urlsafe(5)
    db.execute("INSERT INTO links(code,url) VALUES(?,?)",(code,url)); db.commit()
    return jsonify({"code":code,"short_url":request.host_url+code})

@app.get("/<code>")
def go(code):
    row=db.execute("SELECT url FROM links WHERE code=?",(code,)).fetchone()
    if not row: return "Not found",404
    db.execute("UPDATE links SET clicks=clicks+1 WHERE code=?",(code,)); db.commit()
    return redirect(row[0])

@app.get("/stats/<code>")
def stats(code):
    row=db.execute("SELECT code,url,clicks FROM links WHERE code=?",(code,)).fetchone()
    return jsonify(dict(zip(["code","url","clicks"],row))) if row else ("Not found",404)

app.run(debug=True)
```

7. Weather Dashboard

CLI weather dashboard using Open-Meteo with caching and forecast summaries.

```
import requests, json, time
from pathlib import Path

city=input("City: ")
geo=requests.get("https://geocoding-api.open-meteo.com/v1/search",
                 params={"name":city,"count":1,"language":"en","format":"json"}).json()
if not geo.get("results"): raise SystemExit("City not found")
g=geo["results"][0]
params={"latitude":g["latitude"],"longitude":g["longitude"],
        "current":"temperature_2m,relative_humidity_2m,wind_speed_10m",
        "daily":"temperature_2m_max,temperature_2m_min,precipitation_probability_max",
        "timezone":"auto"}
data=requests.get("https://api.open-meteo.com/v1/forecast",params=params).json()
c=data["current"]
print(f"\n{g['name']}, {g.get('country','')}")
print(f"Now: {c['temperature_2m']}°C | Humidity: {c['relative_humidity_2m']}%")
print(f"Wind: {c['wind_speed_10m']} km/h")
for d,hi,lo,rain in zip(data["daily"]["time"],data["daily"]["temperature_2m_max"],
                        data["daily"]["temperature_2m_min"],data["daily"][
"precipitation_probability_max"]):
    print(d, f"High {hi}°C Low {lo}°C Rain {rain}%")
```

8. Contact Book with Search

SQLite contact manager with tags, search, update and JSON backup.

```
import sqlite3, json
db=sqlite3.connect("contacts.db")
db.execute("CREATE TABLE IF NOT EXISTS contacts(id INTEGER PRIMARY KEY,name TEXT,phone TEXT,email
TEXT,tags TEXT)")
db.commit()

def add():
    v=[input(x+": ") for x in ("Name","Phone","Email","Tags")]
    db.execute("INSERT INTO contacts(name,phone,email,tags) VALUES(?,?,?,?)",v); db.commit()

def search():
    q=input("Search: ")
    rows=db.execute("""SELECT * FROM contacts
      WHERE name LIKE ? OR phone LIKE ? OR email LIKE ? OR tags LIKE ?""",
      tuple("%"+q+"%" for _ in range(4))).fetchall()
    for r in rows: print(r)

def backup():
    rows=db.execute("SELECT name,phone,email,tags FROM contacts").fetchall()
    json.dump([dict(zip(["name","phone","email","tags"],r)) for r in rows],
              open("contacts_backup.json","w"),indent=2)

while True:
    print("\n1 Add 2 Search 3 Backup 4 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": search()
    elif c=="3": backup(); print("Backup saved.")
    elif c=="4": break
```

9. Study Planner

Priority-based study planner with deadlines, completion tracking and daily agenda.

```
import sqlite3
from datetime import date

db=sqlite3.connect("study.db")
db.execute("""CREATE TABLE IF NOT EXISTS tasks(
id INTEGER PRIMARY KEY,subject TEXT,task TEXT,due TEXT,priority INTEGER,done INTEGER DEFAULT 0)""")
db.commit()

def add():
    s=input("Subject: "); t=input("Task: "); d=input("Due YYYY-MM-DD: ")
    p=int(input("Priority 1-5: "))
    db.execute("INSERT INTO tasks(subject,task,due,priority) VALUES(?,?,?,?)",(s,t,d,p)); db.commit(
)

def agenda():
    rows=db.execute("""SELECT id,subject,task,due,priority FROM tasks
                       WHERE done=0 ORDER BY due,priority DESC""").fetchall()
    for r in rows: print(r)

def done():
    i=input("Task id: ")
    db.execute("UPDATE tasks SET done=1 WHERE id=?",(i,)); db.commit()

while True:
    print("\n1 Add 2 Agenda 3 Complete 4 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": agenda()
    elif c=="3": done()
    else: break
```

10. Inventory Management System

SQLite inventory with stock alerts, purchase entries and sales deductions.

```
import sqlite3
db=sqlite3.connect("inventory.db")
db.execute("""CREATE TABLE IF NOT EXISTS items(
sku TEXT PRIMARY KEY,name TEXT,stock INTEGER,price REAL,reorder INTEGER)""")
db.commit()

def add_item():
    sku=input("SKU: "); name=input("Name: "); stock=int(input("Stock: "))
    price=float(input("Price: ")); reorder=int(input("Reorder level: "))
    db.execute("INSERT OR REPLACE INTO items VALUES(?,?,?,?,?)",(sku,name,stock,price,reorder)); db.
commit()

def sell():
    sku=input("SKU: "); qty=int(input("Qty: "))
    row=db.execute("SELECT stock FROM items WHERE sku=?",(sku,)).fetchone()
    if not row or row[0]<qty: return print("Insufficient stock")
    db.execute("UPDATE items SET stock=stock-? WHERE sku=?",(qty,sku)); db.commit()

def report():
    for r in db.execute("SELECT sku,name,stock,price,reorder FROM items ORDER BY stock"):
        flag=" LOW" if r[2]<=r[4] else ""
        print(r,flag)

while True:
    print("\n1 Add/Update 2 Sell 3 Report 4 Exit")
    c=input("> ")
    if c=="1": add_item()
    elif c=="2": sell()
    elif c=="3": report()
    else: break
```

11. Pomodoro Productivity Desktop

GUI productivity timer with session history.

```
import sqlite3, json
from datetime import datetime

DB="pomodoro_productivity_desktop.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("pomodoro_productivity_desktop.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

12. Markdown Notes Manager

Searchable markdown notebook with tags and index.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

13. Habit Tracker

SQLite habit tracker with streak calculations.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

14. Banking Simulator

Account management, transfers, transaction history and statements.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("banking_simulator.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

15. Library Management System

Books, members, lending, returns and overdue reports.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

16. Hotel Booking Manager

Rooms, reservations, check-in/check-out and billing.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

17. Cinema Ticket Booking

Seat map, bookings, cancellation and receipt generation.

```
import sqlite3, json
from datetime import datetime

DB="cinema_ticket_booking.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("cinema_ticket_booking.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

18. Restaurant POS

Menu, cart, taxes, discounts and daily sales report.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

19. Invoice & GST Calculator

Invoice builder with Indian GST breakdown and PDF export.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

20. Attendance Management

Student attendance database with percentage reports.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("attendance_management.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

21. Employee Payroll System

Salary calculation, deductions, payslip generation.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

22. Resume Builder

JSON-driven resume builder exporting polished PDF.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

23. Job Application Tracker

Track companies, stages, deadlines and follow-ups.

```
import sqlite3, json
from datetime import datetime

DB="job_application_tracker.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("job_application_tracker.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

24. Interview Question Bank

Searchable categorized interview questions with quiz mode.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

25. DSA Practice Tracker

Track problems, difficulty, patterns and revision dates.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

26. Code Snippet Manager

Tagged snippet vault with full-text search.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("code_snippet_manager.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

27. Git Commit Analyzer

Analyze a local Git repository and produce contribution reports.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

28. Log File Analyzer

Parse application logs and generate error summaries.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

29. Website Uptime Monitor

Periodically check URLs and save availability history.

```
import sqlite3, json
from datetime import datetime

DB="website_uptime_monitor.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("website_uptime_monitor.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

30. Broken Link Checker

Crawl a website and report broken internal links.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

31. Website Sitemap Generator

Crawl internal pages and create sitemap.xml.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

32. Web Scraper to CSV

Extract structured product/article data into CSV.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("web_scraper_to_csv.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

33. News Aggregator

Fetch RSS feeds, deduplicate and categorize headlines.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

34. Email Newsletter Builder

Build HTML newsletters from templates and data.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

35. QR Code Event Check-in

Generate attendee QR codes and scan/check-in records.

```
import sqlite3, json
from datetime import datetime

DB="qr_code_event_check_in.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("qr_code_event_check_in.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

36. PDF Merger & Splitter

GUI tool for merging and splitting PDF documents.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

37. Image Batch Processor

Resize, watermark and compress folders of images.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

38. Duplicate Photo Finder

Perceptual hashing to find visually similar photos.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("duplicate_photo_finder.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

39. Video Metadata Catalog

Catalog video metadata and searchable library.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

40. Audio Playlist Manager

Manage playlists and generate M3U files.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

41. YouTube Download Queue

Queue public video URLs for local processing.

```
import sqlite3, json
from datetime import datetime

DB="youtube_download_queue.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("youtube_download_queue.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

42. Personal Knowledge Base

Local searchable notes with backlinks.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

43. Password Strength Auditor

Score passwords locally with entropy rules.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

44. OTP Authenticator Demo

Generate TOTP codes for locally stored demo secrets.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("otp_authenticator_demo.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

45. Secure File Locker

Encrypt/decrypt user-selected files with a password.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

46. Clipboard History Manager

Desktop clipboard history with search.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

47. Screenshot Organizer

Capture screenshots and auto-classify by date.

```
import sqlite3, json
from datetime import datetime

DB="screenshot_organizer.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("screenshot_organizer.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

48. Desktop Notification Scheduler

Schedule local reminders and notifications.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

49. System Resource Dashboard

Monitor CPU, RAM, disk and network usage.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

50. Process Manager

List, filter and safely request termination of processes.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("process_manager.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

51. Disk Usage Analyzer

Recursive folder size analyzer with largest-items report.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

52. Backup Manager

Versioned folder backups with retention policy.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

53. Folder Sync Tool

One-way sync with hashes and dry-run preview.

```
import sqlite3, json
from datetime import datetime

DB="folder_sync_tool.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("folder_sync_tool.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

54. CSV Cleaner

Detect missing values, duplicates and malformed rows.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

55. Excel Report Generator

Create formatted Excel reports from CSV data.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

56. PDF Report Generator

Turn analytics data into a formatted PDF report.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("pdf_report_generator.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

57. Sales Dashboard

Plot sales trends, regions and top products.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

58. Student Performance Dashboard

Analyze marks, grades and subject trends.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

59. Loan EMI Planner

Calculate EMI, amortization schedule and payoff scenarios.

```
import sqlite3, json
from datetime import datetime

DB="loan_emi_planner.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("loan_emi_planner.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

60. Investment Portfolio Tracker

Track holdings, average cost and P/L from CSV prices.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

61. Crypto Portfolio Simulator

Paper portfolio with simulated trades and P/L.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

62. Currency Converter

Live exchange-rate converter with cached rates.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("currency_converter.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

63. Flight Fare Watcher

Store target routes and compare fetched public fare data.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

64. Product Price Tracker

Track public product prices from supplied pages.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

65. Book Recommendation Engine

Content-based recommendation from book metadata.

```
import sqlite3, json
from datetime import datetime

DB="book_recommendation_engine.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("book_recommendation_engine.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

66. Movie Recommendation Engine

Genre/tag similarity recommendation system.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

67. Music Recommendation Engine

Similarity-based playlist generator.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

68. Resume Keyword Matcher

Compare a resume against a job description.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("resume_keyword_matcher.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

69. Cover Letter Generator

Template-driven cover letters from structured data.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

70. Text Summarizer

Extractive summarizer for local text documents.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

71. PDF Text Search Engine

Index PDF text and search across a local collection.

```
import sqlite3, json
from datetime import datetime

DB="pdf_text_search_engine.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("pdf_text_search_engine.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

72. OCR Document Pipeline

OCR image documents and save searchable text.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

73. Language Detector

Detect language from text using a lightweight model.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

74. Sentiment Analyzer

Classify review sentiment with TF-IDF + logistic regression.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("sentiment_analyzer.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

75. Spam Email Classifier

Train a local text classifier on labeled emails.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

76. House Price Predictor

Regression model with preprocessing and evaluation.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

77. Student Score Predictor

Regression model with train/test evaluation.

```
import sqlite3, json
from datetime import datetime

DB="student_score_predictor.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("student_score_predictor.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

78. Customer Churn Predictor

Classification pipeline with metrics and feature importance.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

79. Loan Approval Predictor

Classification demo with preprocessing and metrics.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

80. Sales Forecasting Tool

Time-series baseline forecast and visualization.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("sales_forecasting_tool.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

81. Anomaly Detector

Detect unusual numeric observations with robust statistics.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

82. Image Classifier

Train a small CNN on a folder-based image dataset.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

83. Face Attendance Demo

Face embeddings for local attendance demonstration.

```
import sqlite3, json
from datetime import datetime

DB="face_attendance_demo.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("face_attendance_demo.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

84. Hand Gesture Controller

Recognize basic hand gestures with webcam input.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

85. Object Detection Demo

Run a pretrained detector on webcam frames.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

86. Voice Notes App

Record microphone audio and create timestamped notes.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("voice_notes_app.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

87. Speech-to-Text Journal

Transcribe microphone input into daily journal files.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

88. Text-to-Speech Reader

Read documents aloud with playback controls.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

89. AI Chat Assistant

Local chat UI with pluggable model/API backend.

```
import sqlite3, json
from datetime import datetime

DB="ai_chat_assistant.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("ai_chat_assistant.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

90. Document Q&A; Assistant

Chunk local documents and retrieve relevant passages.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

91. Code Explanation Assistant

Generate structured explanations for pasted code.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

92. SQL Query Assistant

Convert natural-language questions into reviewed SQL.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("sql_query_assistant.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

93. AI Study Buddy

Generate quizzes from user-provided study notes.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

94. Flashcard Generator

Turn notes into structured flashcards.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

95. Timetable Generator

Constraint-based class timetable generator.

```
import sqlite3, json
from datetime import datetime

DB="timetable_generator.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("timetable_generator.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```

96. Pathfinding Visualizer

Visualize BFS, DFS and A* on a grid.

```
from pathlib import Path
import json, csv, re

ROOT=Path(".")
query=input("Search term: ").lower()
extensions={".txt",".md",".py",".csv",".json",".html",".js",".css"}
results=[]
for p in ROOT.rglob("*"):
    if p.is_file() and p.suffix.lower() in extensions:
        try: text=p.read_text(errors="ignore")
        except: continue
        score=text.lower().count(query)
        if score:
            results.append((score,p))
for score,p in sorted(results,reverse=True):
    print(f"{score:4}  {p}")
print(f"\nFound {len(results)} matching files.")
```

97. Sorting Visualizer

Animate multiple sorting algorithms with metrics.

```
import csv, statistics
from collections import Counter,defaultdict

FILE=input("CSV file: ")
with open(FILE,newline="",encoding="utf-8-sig") as f:
    rows=list(csv.DictReader(f))
print("Rows:",len(rows))
print("Columns:",list(rows[0]) if rows else [])
for col in (rows[0] if rows else {}):
    values=[r[col] for r in rows]
    missing=sum(v.strip()=="" for v in values)
    print(f"{col}: missing={missing}, unique={len(set(values))}")
# Duplicate report
keys=[tuple(r.values()) for r in rows]
dups=Counter(keys)
print("Duplicate rows:",sum(n-1 for n in dups.values() if n>1))
```

98. Tic-Tac-Toe AI

Minimax-based unbeatable Tic-Tac-Toe.

```
from flask import Flask, request, jsonify
import sqlite3

app=Flask(__name__)
db=sqlite3.connect("tic_tac_toe_ai.db",check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS items(id INTEGER PRIMARY KEY,title TEXT,body TEXT,status
TEXT)")
db.commit()

@app.get("/items")
def all_items():
    rows=db.execute("SELECT id,title,body,status FROM items ORDER BY id DESC").fetchall()
    return jsonify([dict(zip(["id","title","body","status"],r)) for r in rows])

@app.post("/items")
def create():
    x=request.json or {}
    db.execute("INSERT INTO items(title,body,status) VALUES(?,?,?)",
               (x.get("title",""),x.get("body",""),x.get("status","open")))
    db.commit()
    return jsonify({"ok":True}),201

@app.patch("/items/<int:item_id>")
def update(item_id):
    x=request.json or {}
    db.execute("UPDATE items SET title=COALESCE(?,title),body=COALESCE(?,body),status=COALESCE(?,
status) WHERE id=?",
               (x.get("title"),x.get("body"),x.get("status"),item_id))
    db.commit()
    return jsonify({"updated":db.total_changes})

@app.delete("/items/<int:item_id>")
def delete(item_id):
    db.execute("DELETE FROM items WHERE id=?",(item_id,)); db.commit()
    return jsonify({"deleted":item_id})

app.run(debug=True)
```

99. Snake Game

Pygame snake with levels, score and saved high score.

```
import csv, math
from collections import defaultdict

FILE=input("Data CSV: ")
with open(FILE,newline="",encoding="utf-8-sig") as f: rows=list(csv.DictReader(f))
if not rows: raise SystemExit("CSV is empty")

print("\nDataset summary")
for k in rows[0]:
    vals=[]
    for r in rows:
        try: vals.append(float(r[k]))
        except: pass
    if vals:
        mean=sum(vals)/len(vals)
        var=sum((x-mean)**2 for x in vals)/len(vals)
        print(f"{k:20} n={len(vals):5} mean={mean:.2f} min={min(vals):.2f} max={max(vals):.2f} sd={
math.sqrt(var):.2f}")
    else:
        counts=defaultdict(int)
        for r in rows: counts[r[k]]+=1
        print(f"{k:20} categories={len(counts)} top={sorted(counts.items(),key=lambda x:-x[1])[:
5]}")
```

100. Flappy Bird Clone

Pygame arcade game with collision and score.

```
import argparse, shutil, hashlib
from pathlib import Path

parser=argparse.ArgumentParser()
parser.add_argument("source",type=Path); parser.add_argument("backup",type=Path)
parser.add_argument("--dry-run",action="store_true")
args=parser.parse_args()

def digest(p):
    h=hashlib.sha256()
    with open(p,"rb") as f:
        for c in iter(lambda:f.read(1024*1024),b""): h.update(c)
    return h.hexdigest()

for src in args.source.rglob("*"):
    if not src.is_file(): continue
    dst=args.backup/src.relative_to(args.source)
    dst.parent.mkdir(parents=True,exist_ok=True)
    if dst.exists() and digest(src)==digest(dst):
        continue
    print(src,"->",dst)
    if not args.dry_run: shutil.copy2(src,dst)
print("Backup complete.")
```

101. Space Shooter

Pygame shooter with waves, power-ups and scoring.

```
import sqlite3, json
from datetime import datetime

DB="space_shooter.db"
db=sqlite3.connect(DB)
db.execute("CREATE TABLE IF NOT EXISTS records(id INTEGER PRIMARY KEY, title TEXT, status TEXT,
priority INTEGER, created TEXT)")
db.commit()

def add():
    title=input("Title: "); status=input("Status [todo]: ") or "todo"
    priority=int(input("Priority 1-5: ") or 3)
    db.execute("INSERT INTO records(title,status,priority,created) VALUES(?,?,?,?)",
               (title,status,priority,datetime.now().isoformat(timespec="seconds"))); db.commit()

def list_items():
    for r in db.execute("SELECT id,title,status,priority,created FROM records ORDER BY priority
DESC,created DESC"):
        print(r)

def update():
    i=input("ID: "); s=input("New status: ")
    db.execute("UPDATE records SET status=? WHERE id=?",(s,i)); db.commit()

while True:
    print("\n1 Add  2 List  3 Update  4 Export  5 Exit")
    c=input("> ")
    if c=="1": add()
    elif c=="2": list_items()
    elif c=="3": update()
    elif c=="4":
        rows=db.execute("SELECT * FROM records").fetchall()
        json.dump(rows,open("space_shooter.json","w"),indent=2,default=str)
        print("Exported.")
    elif c=="5": break
```